第A8讲 消息传递编程: MPI

何克晶

kejinghe@gmail.com

华南理工大学

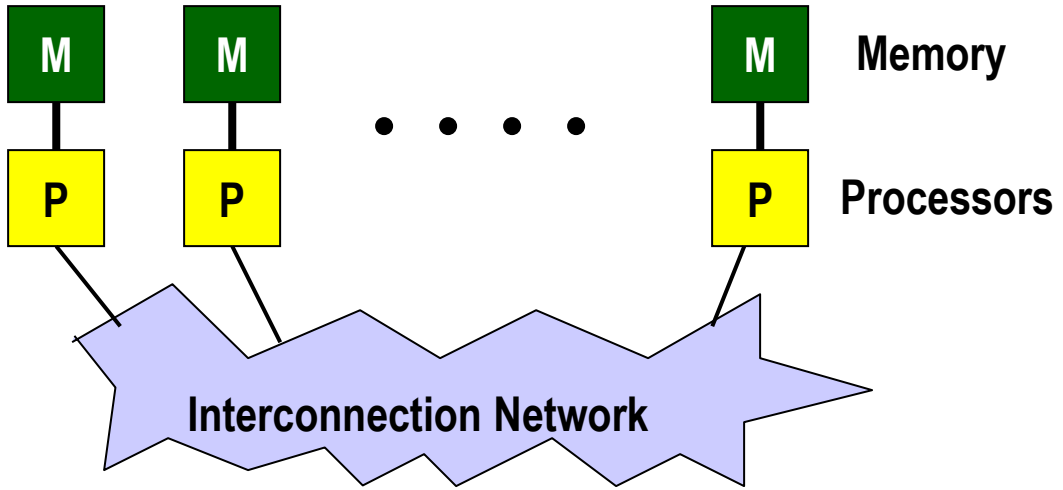
计算机科学与工程学院

Outline

- Message Passing Programming Overview
- Parallel Programming with MPI
- MPI Programming Example

Message-Passing Programming Paradigm

- Each processor in a message-passing program runs a sub-program
  - written in a conventional sequential language
  - all variables are private
  - communicate via special subroutine calls



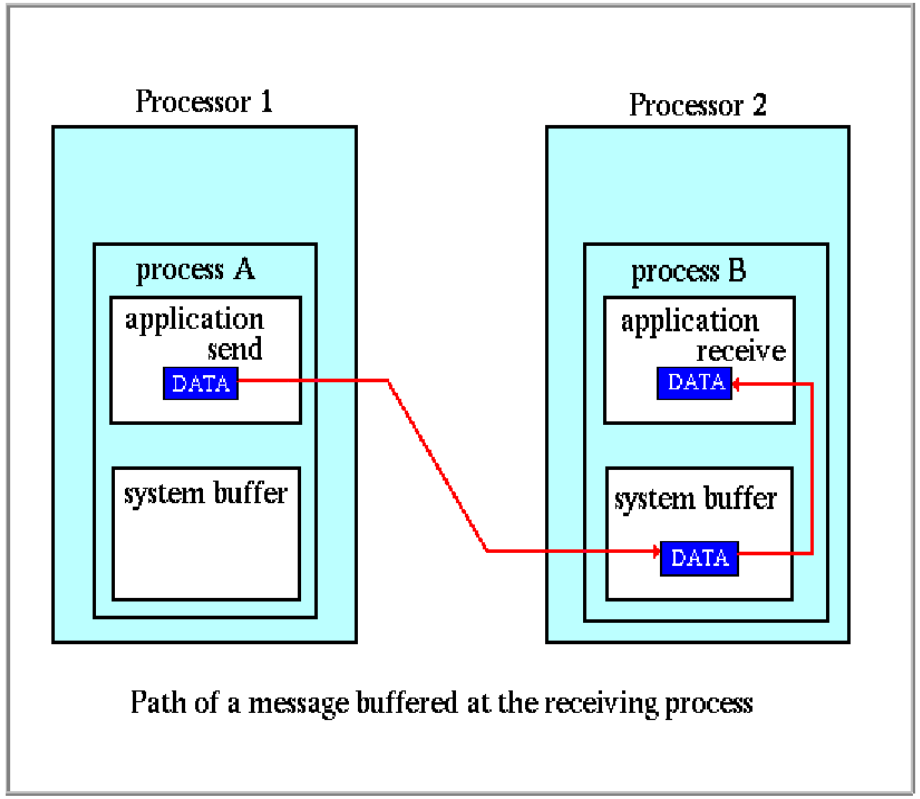
Message-Passing Programming Overview

- Model
  - Set of processes that each have local data and are able to communicate with each other by sending and receiving messages
  - Separate processes, separate address spaces (distributed memory model)
  - Processes execute independently and concurrently, and transfer data cooperatively

Messages

- Messages are packets of data moving between sub-programs

- The message passing system has to be told the following information
  - Sending processor
  - Source location
  - Data type
  - Data length
  - Receiving processor(s)
  - Destination location
  - Destination size



Path of a message buffered at the receiving process

Messages (Cont.)

- Access:
  - Each sub-program needs to be connected to a message passing system
- Addressing:
  - Messages need to have addresses to be sent to
- Reception:
  - It is important that the receiving process is capable of dealing with the messages it is sent
- A message passing system is similar to:
  - Post-office, Phone line, Fax, E-mail, etc
- Communication Types:
  - Point-to-Point, Collective, Synchronous /Asynchronous

Point-to-Point Communication

- Simplest form of message passing: one process sends a message to another
- Several variations on how sending a message can interact with execution of the sub-program
  - Synchronous Sends
    - provide information about the completion of the message
    - e.g. telephone, fax machines
  - Asynchronous Sends
    - Only know when the message has left
    - e.g. post cards
  - Blocking operations
    - only return from the call when operation has completed
  - Non-blocking operations
    - return straight away - can test/wait later for completion

Collective Communications

- Collective communication routines are higher level routines involving several processes at a time
- Can be built out of point-to-point communications
- Barriers
  - synchronise processes
- Broadcast
  - one-to-many communication
- Reduction operations
  - combine data from several processes to produce a single (usually) result

Message Passing Systems

- Initially each manufacturer developed their own, wide range of features, often incompatible
- PVM - Parallel Virtual Machine
  - *de facto* standard before MPI
  - Open Source (NOT public domain!)
  - User Interface to the System (daemons)
  - Support for Dynamic environments
- MPI - Message Passing Interface
  - provide source-code portability
  - allow efficient implementation
  - provide a high level of functionality
  - support for heterogeneous parallel architectures

MPI History

- MPI Forum (www.mpi-forum.org): government, industry and academia.
  - Formal process began November 1992. Sixty people from forty different organisations.
  - Draft presented at Supercomputing 1993
  - Final standard (1.0) ,115 routines/functions that can be grouped into 8 categories, published May 1994
  - Clarifications (1.1) published June1995
  - MPI-2 process began April, 1995
  - MPI-1.2 finalized July 1997
  - MPI-2 finalized July 1997

MPI Implementations

- MPICH from Argonne National Lab and Mississippi State Univ.
  - http://www.mcs.anl.gov/mpi/mpich
- LAM (Local Area Multicomputer), developed at the Ohio Supercomputer Center
  - http://www.lam-mpi.org/
- Others:
  - MS MPI, Intel MPI
  - OpenMPI, CHIMP

Outline

- Message Passing Programming Overview
- Parallel Programming with MPI
- MPI Programming Example

General MPI Program Structure

MPI Include File

Initialize MPI Environment

Do work and perform message communication

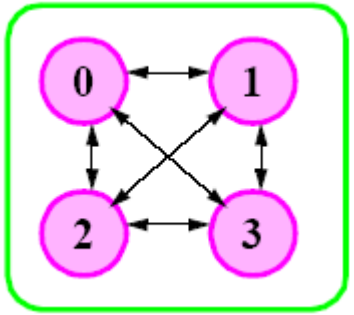
Terminate MPI Environment

MPI Programming Overview

1. Creating parallelism
   - SPMD Model
2. Communication between processors
   - Basic
   - Collective
   - Non-blocking
3. Synchronization
   - Point-to-point synchronization is done by message passing
   - Global synchronization done by collective communication

An MPI Application

- An MPI application



- The elements of the application are:
  - 4 processes, numbered zero through three
  - Communication paths between them
- The set of processes plus the communication channels is called
  - "MPI_COMM_WORLD"

Example: Hello World

```
#include "mpi.h"
#include <stdio.h>

   int main( int argc, char *argv[] )
   {
       int rank, size, namelen;
       char processor_name[MPI_MAX_PROCESSOR_NAME];

       MPI_Init( &argc, &argv );
       MPI_Comm_rank( MPI_COMM_WORLD, &rank );
       MPI_Comm_size( MPI_COMM_WORLD, &size );
       MPI_Get_processor_name(processor_name, &namelen);
       printf("Hello World! I'm rank %d of %d on %s\n",
              rank, size, processor_name);

       MPI_Finalize();
       return 0;
   }
```

Function Call Explanations

- `MPI_Init()` starts MPI running, creates a set of parallel processes
- `MPI_Comm_size()` returns the number of processes in the communicator group `MPI_COMM_WORLD`
- The rank of the calling process (ranging from 0 to *nprocs - 1*) is provided by `MPI_Comm_rank()`
- `Get_processor_name()` provides the hostname of the node (not the individual processor) that is being used
- `MPI_Finalize()` is called to terminate the parallel environment

Compiling and Running

- Compile (may be different on every machine):

  mpicc -o hello hello.c

- Start four processes (somewhere):

  mpirun -np 4 ./hello

- Run with 4 processes:

  Hello World! I'm rank 0 of 4 on node01

  Hello World! I'm rank 2 of 4 on node03

  Hello World! I'm rank 3 of 4 on node04

  Hello World! I'm rank 1 of 4 on node02

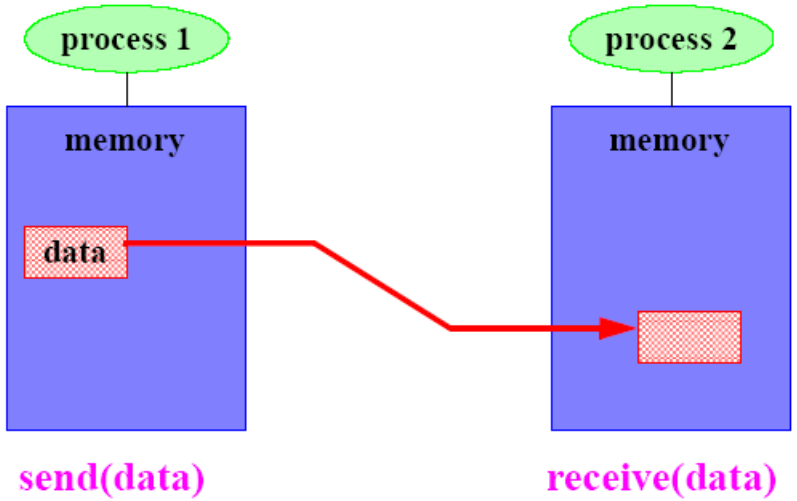
- Note:
  - Order of output is not specified by MPI

Outline

- Message Passing Programming Overview
- Parallel Programming with MPI
  - Point-to-Point Communication
  - Collective Communication
  - Communication Modes
- MPI Programming Example

MPI Basic Send/Receive

- "Two sided" – both sender and receiver must take action.



- Things that need specifying:
  - How will processes be identified?
  - How will data be described?
  - How will the receiver recognize/screen messages?
  - What will it mean for these operations to complete?

Send & Receive

- MPI_SEND : performs a send operation
  - int MPI_Send(void *buf, int count, MPI_Datatype dataytpe, int dest, int tag, MPI_Comm comm)
  - Message Buffer: (buf, count, datatype)
  - Message Envelop: (dest, tag, comm)
- MPI_RECV : performs a receive operation
  - int MPI_Recv(void *buf, int count, MPI_Datatype datatyepe,int source, int tag, MPI_Comm comm, MPI_Status *status)

Identifying Processes: MPI Communicators

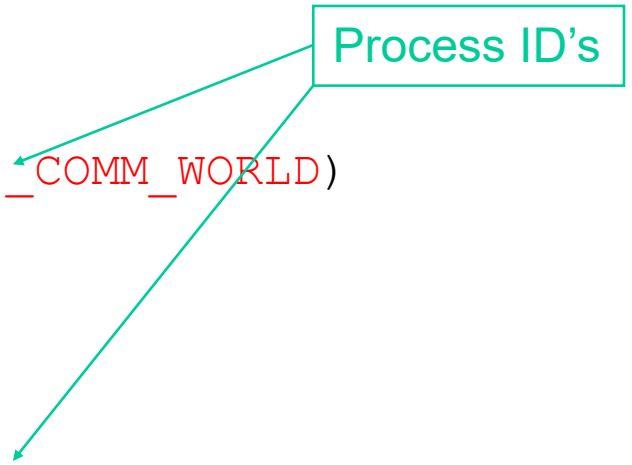
- Processes can be subdivided into groups:
  - A process can be in many groups
  - Groups can overlap
- Supported using a "communicator (通信体):" a message *context* and a *group* of processes
  - A group of processes (进程组)
    - Numbered 0 … N-1
    - Never changes membership
  - Context (进程活动环境): a set of private communication channels between them
    - Message sent with one communicator cannot be received by another.
    - Implemented using hidden message tag
- In a simple MPI program all processes do the same thing:
  - The set of all processes make up the "world":
    - MPI_COMM_WORLD
  - Name processes by number (called "rank")

Point-to-Point Communication Example

Process 0 sends array “A” to process 1 which receives it as “B”

```
1:
  #define TAG 123
  double A[10];
  MPI_Send(A, 10, MPI_DOUBLE, 1, TAG, MPI_COMM_WORLD)

2:
  #define TAG 123
  double B[10];
  MPI_Recv(B, 10, MPI_DOUBLE, 0, TAG, MPI_COMM_WORLD,    &status)
or
  MPI_Recv(B, 10, MPI_DOUBLE, MPI_ANY_SOURCE,
        MPI_ANY_TAG, MPI_COMM_WORLD, &status)
```



Describing Data: MPI Datatypes

- The data in a message to be sent or received is described by a triple (address, count, datatype), where
- An MPI *datatype* is recursively defined as:
  - predefined, corresponding to a data type from the language (e.g., MPI_INT, MPI_DOUBLE_PRECISION)
  - Aggregate Type
    - Vector
    - Struct
    - Indexed
- There are MPI functions to construct custom datatypes, such an array of (int, float) pairs, or a row of a matrix stored columnwise.

MPI Predefined Datatypes

- MPI_INT (int)
- MPI_FLOAT (float)
- MPI_DOUBLE (double)
- MPI_CHAR (signed char)
- MPI_LONG (long)
- MPI_SHORT (short)
- MPI_UNSIGNED (unsigned int)
- MPI_UNSIGNED_LONG (unsigned long)
- MPI_UNSIGNED_SHORT (unsigned short)

- MPI_BYTE
- MPI_PACKED

Recognizing & Screening Messages: MPI Tags

- Messages are sent with a user-defined integer tag:
  - Allows receiving process in identifying the message.
  - Receiver may also screen messages by specifying a tag.
  - Use `MPI_ANY_TAG` to avoid screening.

Process P:	**Process Q:**
Send(A, 32, Q)	**recv (X, 32, P)**
Send(B, 16, Q)	**recv (Y, 16, P)**

Source/Destination

- Destination
  - Rank of process message is being sent to (destination)
  - Must be a valid rank (0...N-1) in communicator
- Source
  - Rank of process message is being received from (source)
  - "Wildcard" `MPI_ANY_SOURCE` matches any source

Message Status

- `Status` is a data structure
  - `status.MPI_TAG` is tag of incoming message
  - `status.MPI_SOURCE` is source of incoming message
    - How many elements of given datatype were received
  - `MPI_Get_count`(IN status, IN datatype, OUT count)
    - Especially useful with wild-cards to find out what matched:

```
int recvd_tag, recvd_from, recvd_count;
MPI_Status status;

MPI_Recv(..., MPI_ANY_SOURCE, MPI_ANY_TAG, ...,
   &status )

recvd_tag  = status.MPI_TAG;
recvd_from = status.MPI_SOURCE;

MPI_Get_count( &status, datatype, &recvd_count );
```

MPI Basic (Blocking) Send

- `MPI_SEND (start, count, datatype, dest, tag, comm)`
  - start: a pointer to the start of the data
  - count: the number of elements to be sent
  - datatype: the type of the data
  - dest: the rank of the destination process
  - tag: the tag on the message for matching
  - comm: the communicator to be used.

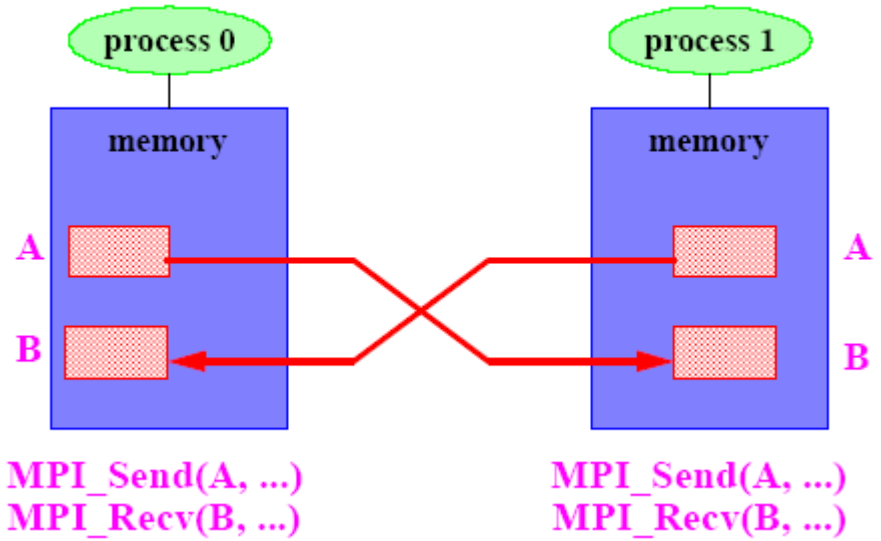
- Completion: When this function returns, the data has been delivered to the "system" and the data structure (start…start+count) can be reused. The message may not have been received by the target process.

Summary of Basic Point-to-Point MPI

- Many parallel programs can be written using just these functions:
  - `MPI_INIT`
  - `MPI_FINALIZE`
  - `MPI_COMM_SIZE`
  - `MPI_COMM_RANK`
  - `MPI_SEND`
  - `MPI_RECV`
  - `MPI_SENDRECV`
- Point-to-point (send/recv) isn't the only way...

Non-Blocking Communication

- Blocking & Non-blocking
  - Blocking stops the program until the message buffer is safe to use
  - Non-blocking separates communication from computation
- The following is called an “unsafe” MPI program



It may run or not, depending on the availability of system buffers to store the messages.

Non-blocking Operations

- Split communication operations into two parts.
  - First part initiates the operation. It does not block.
  - Second part waits for the operation to complete.

```
MPI_Request request;
    MPI_Recv(buf, count, type, dest, tag, comm, status)
      =
    MPI_Irecv(buf, count, type, dest, tag, comm, &request)
      +
    MPI_Wait(&request, &status)

    MPI_Send(buf, count, type, dest, tag, comm)
      =
    MPI_Isend(buf, count, type, dest, tag, comm, &request)
      +
    MPI_Wait(&request, &status)
```

Using Non-blocking Receive

- Two advantages:
  - No deadlock (correctness)
  - Data may be transferred concurrently (performance)

```
 #define MYTAG 123
  #define WORLD MPI_COMM_WORLD
  MPI_Request request;
  MPI_Status status;
Process 0:
  MPI_Irecv(B, 100, MPI_DOUBLE, 1, MYTAG, WORLD, &request)
  MPI_Send(A, 100, MPI_DOUBLE, 1, MYTAG, WORLD)
  MPI_Wait(&request, &status)
Process 1:
  MPI_Irecv(B, 100, MPI_DOUBLE, 0, MYTAG, WORLD, &request)
  MPI_Send(A, 100, MPI_DOUBLE, 0, MYTAG, WORLD)
  MPI_Wait(&request, &status)
```

Outline

- Message Passing Programming Overview
- Parallel Programming with MPI
  - Basic Point-to-Point Communication
  - Collective Communication
  - Communication mode
- MPI Programming Example

Model for Collective Communication

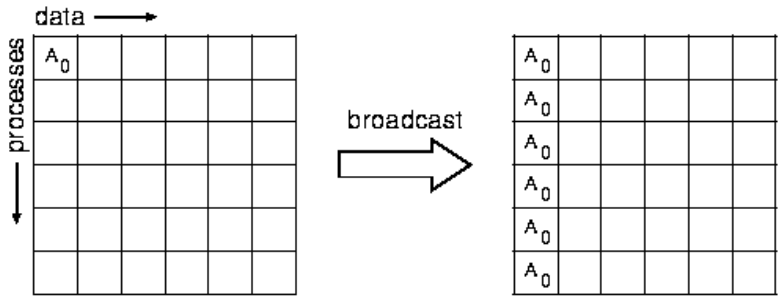
- All processes in communicator must participate
- Process might not finish until all have started.

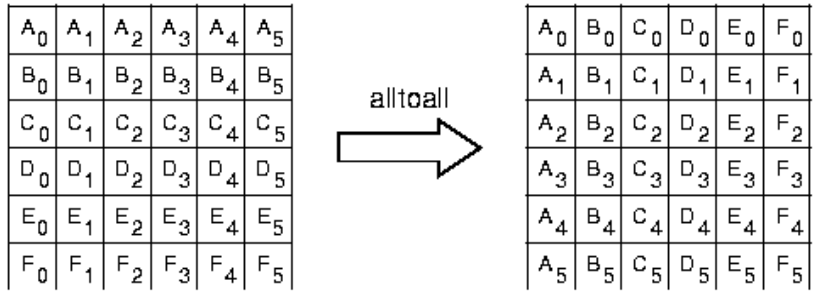
- Functions:
  - Communication: communication among group
  - Synchronization: synchronize the process in group
  - Computation: operation on group data

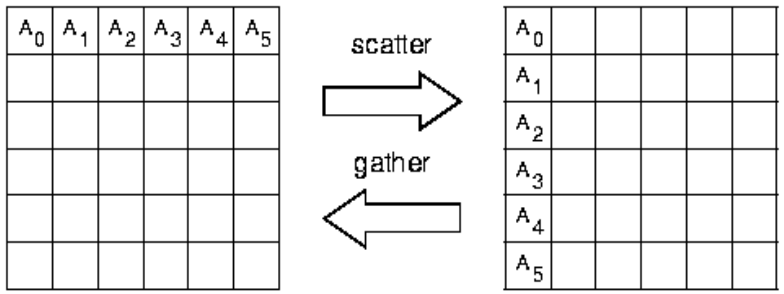
Collective communication

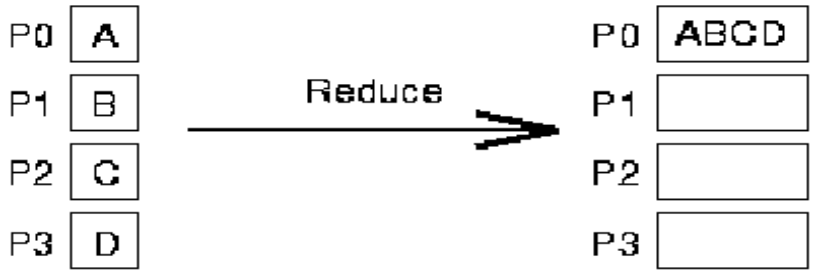
- MPI_Allgather: All processes gather messages
- MPI_Allreduce: Reduce to all processes
- MPI_Alltoall: All processes gather distinct messages
- MPI_Bcast: Broadcast a message
- MPI_Gather: Gather a message to root
- MPI_Scatter: Scatter a message from root
- MPI_Scan: Global prefix reduction
- MPI_Reduce: Global reduce operation
- MPI_ReduceScatter: Reduce and scatter results
- MPI_Barrier: synchronize the process

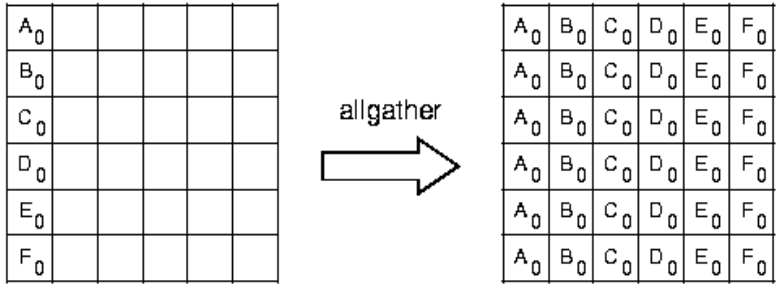
Collective communication (Cont.)

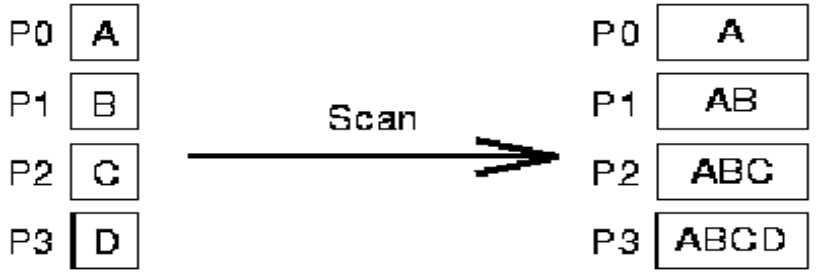












Barrier

- `MPI_Barrier(communicator)`
- No process leaves the barrier until all processes have entered it.

Broadcast

- `MPI_Bcast(buf,len,type,root,comm)`
  - Process with rank = root is source of data (in buf)
  - Other processes receive data

```
MPI_Comm_rank(MPI_COMM_WORLD, &myid);
if (myid == 0) {
/* read data from file */
}
MPI_Bcast(data,len,type,0,MPI_COMM_WORLD);
```

- Note:
  - All processes must participate
  - MPI has no “multicast” that is matched by a receive

Reduction

- Combine elements in input buffer from each process, placing result in output buffer

```
MPI_Reduce(indata,outdata,count,type,op,root,comm)
MPI_Allreduce(indata,outdata,count,type,op,comm)
```

  - Reduce: output appears only in buffer on root
  - Allreduce: output appears on all processes
- operation types:
  - `MPI_SUM`: sum
  - `MPI_PROD`: product
  - `MPI_MAX`: maximum
  - `MPI_MIN`: minimum
  - `MPI_BAND`: bit-wise and
  - Arbitrary user-defined operations on arbitrary user-defined datatypes

Reduction Example: dot product

```
/* distribute two vectors over all processes such that
 processor 0 has elements 0...99
 processor 1 has elements 100...199
 processor 2 has elements 200...299
 etc.
*/

double dotprod(double a[100], double b[100])
{
 double gresult = lresult = 0.0;
 integer i;
 /* compute local dot product */
 for (i = 0; i < 100; i++) lresult += a[i]*b[i];
 MPI_Allreduce(lresult, gresult, 1, MPI_DOUBLE,
     MPI_SUM, MPI_COMM_WORLD);
 return(gresult);
}
```

Data Movement: all-to-all

- All processes send and receive data from all other processes.

```
MPI_Alltoall(sendbuf,sendcount,sendtype,
recvbuf,recvcount,recvtype,comm)
```

- For a communicator with N processes:
  - sendbuf contains N blocks of sendcount elements each
  - recvbuf receives N blocks of recvcount elements each
  - Each process sends block i of sendbuf to process i
  - Each process receives block i of recvbuf from process i
- Example: multidimensional FFT (matrix transpose)

Other Collective Operations

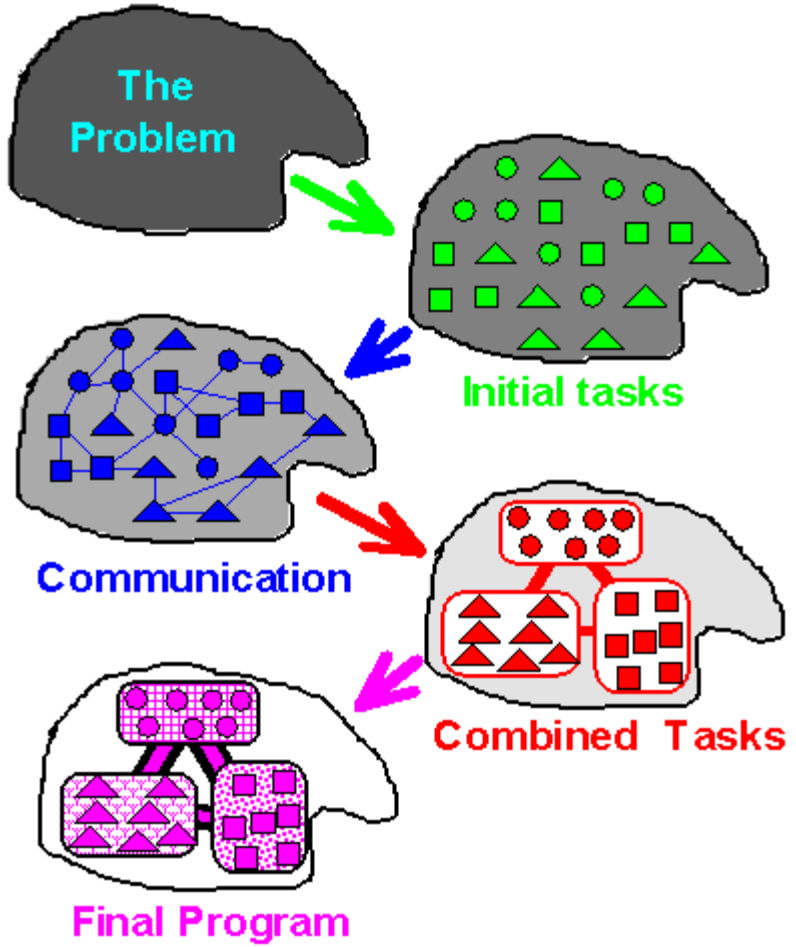
- There are many more collective operations provided by MPI:
- `MPI_Gather/Gatherv/Allgather/Allgatherv`
  - Each process contributes local data that is gathered into a larger array
- `MPI_Scatter/Scatterv`
  - Subparts of a single large array are distributed to processes
- `MPI_Reduce_scatter`
  - Same as Reduce + Scatter
- `Scan`
  - Prefix reduction
- The "v" versions allow processes to contribute different amounts of data

Outline

- Message Passing Programming Overview
- Parallel Programming with MPI
- MPI Programming Example

Designing MPI programs

- Partitioning
  - Before tackling MPI
- Communication
  - Many point to collective operations
- Agglomeration
  - Needed to produce MPI processes
- Mapping
  - Handled by MPI



Example: Calculating PI

```
#include "mpi.h"
#include <math.h>
int main(int argc, char *argv[])
{
 int done = 0, n, myid, numprocs, i, rc;
 double PI25DT = 3.141592653589793238462643;
 double mypi, pi, h, sum, x, a;

 MPI_Init(&argc,&argv);
 MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
 MPI_Comm_rank(MPI_COMM_WORLD,&myid);

 while (!done)  {
   if (myid == 0) {
     printf("Enter the number of intervals: (0 quits) ");
     scanf("%d",&n);
   }
   MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
   if (n == 0) break;
```

$$\pi \approx \frac{1}{n}\sum_{i=1}^{n}\frac{4}{1+\left(\frac{i-0.5}{n}\right)^2}$$

Example: Calculating PI (Cont.)

```
     h   = 1.0 / (double) n;
    sum = 0.0;
    for (i = myid + 1; i <= n; i += numprocs) {
      x = h * ((double)i - 0.5);
      sum += 4.0 / (1.0 + x*x);
    }
    mypi = h * sum;
    MPI_Reduce(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, 0,
               MPI_COMM_WORLD);

    if (myid == 0)
      printf("pi is approximately %.16f, Error is      %.16f\n", pi,
 fabs(pi - PI25DT));

  }
  MPI_Finalize();
   return 0;
}
```

Summary: MPI

- MPI Six Function
  - MPI_Init
  - MPI_Finalize
  - MPI_Comm size
  - MPI_Comm rank
  - MPI_Send
  - MPI_Recv
- MPI Communications
  - Point-to-Point Communication
    - Communicators, datatype, tag, status, source/destination
  - Collective Communication
    - Bcast, Gather, Reduce
  - Non-blocking communication
  - Communication mode
    - Synchronous,Ready, Buffered,Standard

Summary: MPI vs. OpenMP

- Both use SPMD model.
- Message passing v.s. shared data
- Processes v.s. Threads
- MPI has no work sharing structure.

MPI

- Pros:
  - Very portable
  - Requires no special compiler
  - Requires no special hardware but can make use of high performance hardware
  - Very flexible -- can handle just about any model of parallelism
  - No shared data! (You don’t have to worry about processes "treading on each other's data" by mistake.)
  - Can download free libraries for your Linux PC!
  - Forces you to do things the "right way" in terms of decomposing your problem.
- Cons:
  - All-or-nothing parallelism (difficult to incrementally parallelize existing serial codes)
  - No shared data! Requires distributed data structures
  - Could be thought of assembler for parallel computing -- you generally have to write more code
  - Partitioning operations on distributed arrays can be messy.

OpenMP

- Pros:
  - Incremental parallelism -- can parallelize existing serial codes one bit at a time
  - Quite simple set of directives
  - Shared data!
  - Partitioning operations on arrays is very simple.
- Cons:
  - Requires proprietary compilers
  - Requires shared memory multiprocessors
  - Shared data!
  - Having to think about what data is shared and what data is private
  - Cannot handle models like master/slave work allocation (yet)
  - Generally not as scalable (more synchronization points)
  - Not well-suited for non-trivial data structures like linked lists, trees etc

Assigned Readings

- 《并行计算》
  - 第14章：分布式存储系统并行编程
- 《可扩展并行计算》
  - 第13章：消息传递编程

- The MPI Standard：http://www.mpi-forum.org
- Using Microsoft Message Passing Interface
  - http://technet.microsoft.com/en-us/library/cc720120(WS.10).aspx

- 都志辉，高性能计算并行编程技术——MPI并行程序设计，清华大学出版社，2001